

Detection Notes

Windows Detection Engineering Lab — Wazuh, Sysmon & MITRE ATT&CK

Document Type: Detection Engineering Notes

Project: Windows Detection Engineering Lab — Wazuh, Sysmon & MITRE ATT&CK

Detection Scope: 5 validated detections

Environment: Local VirtualBox SOC lab

Tools Used: Wazuh SIEM/XDR, Windows 10, Sysmon, Windows Security Logs, Windows Defender, Atomic Red Team, MITRE ATT&CK

1. Project Objective

The objective of this project is to demonstrate a practical Windows detection engineering workflow using Wazuh SIEM/XDR, Sysmon, Windows Security logs, Windows Defender telemetry, and MITRE ATT&CK mapping.

This project extends the existing Enterprise SOC Lab by moving beyond basic SIEM deployment and focusing specifically on detection creation, validation, evidence collection, false-positive analysis, and analyst triage documentation. The purpose is not just to generate alerts, but to show how endpoint activity can be converted into structured, explainable, and validated detections.

The lab was designed to simulate realistic Windows endpoint activity inside a safe and controlled environment. Each detection was tested using controlled commands or safe simulation tools. The project does not involve real malware execution, credential theft, ransomware simulation, LSASS dumping, command-and-control payloads, or unauthorized testing.

The final validated project scope includes five detections:

Detection ID	Detection Name	Custom Rule ID	MITRE ATT&CK Technique
DET-001	PowerShell Encoded Command Execution	100100	T1059.001

DET-002	Local User Account Creation	100101	T1136.001
DET-003	Windows Service Creation / Modification	100102	T1543.003
DET-009	EICAR Malware-Test File Detection	100108	T1204.002
DET-010	Atomic Red Team Framework Activity	100109	T1059.001

This scope was intentionally kept realistic. Only detections that were tested and supported with Wazuh dashboard evidence were included as validated detections.

2. Lab Architecture Overview

The lab was built using a local VirtualBox environment containing a Wazuh server and a Windows 10 endpoint. The Windows endpoint acted as the primary telemetry source. The Wazuh server received, parsed, and analyzed logs from the Windows endpoint using the Wazuh agent.

The Windows endpoint was configured with Sysmon to provide enhanced process and command-line telemetry. Windows Security logs were used for account-related activity, while Windows Defender Operational logs were used to validate malware-test detection using the EICAR test file.

The architecture followed this detection flow:

Windows 10 Endpoint → Sysmon / Security Logs / Defender Logs → Wazuh Agent → Wazuh Manager → Custom Rules → Wazuh Dashboard → Evidence Screenshots and Documentation

The main lab components were:

Component	Purpose
Windows 10 Endpoint	Generated test activity and endpoint telemetry
Sysmon	Collected process creation and command-line activity
Windows Security Logs	Provided account creation and system security events
Windows Defender Operational Logs	Provided EICAR malware-test detection events

Wazuh Agent	Forwarded Windows telemetry to the Wazuh server
Wazuh Manager	Processed logs, applied rules, and generated alerts
Wazuh Dashboard	Used to review alerts, JSON fields, and MITRE mappings
MITRE ATT&CK	Used to map detections to real-world adversary techniques

This architecture reflects a realistic SOC workflow on a smaller lab scale. Endpoint events are collected, forwarded, decoded, matched against rules, and reviewed inside the SIEM dashboard.

3. Detection Engineering Workflow

The project followed a structured detection engineering workflow. Each detection was treated as a complete detection lifecycle task, not just a one-time command execution.

The workflow used was:

1. Select a suspicious Windows behavior
2. Map the behavior to MITRE ATT&CK
3. Identify the required telemetry source
4. Generate safe test activity in the Windows 10 VM
5. Confirm that Wazuh receives the event
6. Create or validate a custom Wazuh detection rule
7. Trigger the event again after rule implementation
8. Review the alert in Wazuh Dashboard
9. Capture screenshots and JSON evidence
10. Document false-positive scenarios and analyst triage steps

This workflow helped prove that each detection was functional, mapped to a real attack technique, and explainable from an analyst's perspective.

The project focused on five validated detections rather than claiming broad coverage without proof. This makes the project more realistic and credible because every included detection has supporting evidence.

4. Telemetry Sources Used

The project used multiple Windows telemetry sources to create and validate detections.

4.1 Sysmon Telemetry

Sysmon was used to collect detailed Windows endpoint activity, especially process creation events. Sysmon is important because normal Windows logs do not always provide enough command-line visibility for detection engineering.

Sysmon was mainly used for:

- PowerShell process execution
- Command-line arguments
- Atomic Red Team activity
- Process creation events
- Suspicious execution patterns

The most important Sysmon event used in this project was Sysmon Event ID 1, which records process creation activity.

4.2 Windows Security Logs

Windows Security logs were used for account-related activity. These logs are valuable for identifying actions such as local account creation, privilege changes, logon attempts, and other security-relevant events.

In this project, Windows Security logs supported the local user account creation detection.

4.3 Windows Defender Operational Logs

Windows Defender Operational logs were used to validate malware-test detection. The EICAR test file was used as a safe way to trigger Defender without executing real malware.

This proved that Defender security events can be forwarded into Wazuh and reviewed from a SIEM perspective.

4.4 Wazuh Alerts and JSON Evidence

Wazuh Dashboard was used to confirm the detections. For each validated detection, alert details were reviewed through the dashboard and, where available, JSON evidence was collected and sanitized.

The evidence helped confirm:

- The correct rule fired
- The expected host generated the alert
- The relevant event fields were present

- The detection mapped to the intended behavior
 - The alert could be explained during triage
-

5. Rule Development Methodology

The rule development process was based on custom Wazuh rules using project-specific rule IDs. The custom rule IDs were used to clearly separate this project's detection logic from default Wazuh rules.

The validated custom rule IDs were:

Rule ID	Detection
100100	PowerShell encoded command execution
100101	Local user account creation
100102	Windows service creation / modification
100108	EICAR malware-test file detection
100109	Atomic Red Team framework activity

Each rule was designed to detect a specific behavior and was mapped to a MITRE ATT&CK technique. The rules were not presented as production-ready enterprise detections. Instead, they were created and validated as lab-based detection engineering examples.

The project shows the ability to:

- Understand endpoint behavior
 - Identify useful telemetry fields
 - Write custom Wazuh rules
 - Map detections to MITRE ATT&CK
 - Validate alerts using controlled test activity
 - Document analyst-facing investigation logic
-

6. Implemented Detections

DET-001: PowerShell Encoded Command Execution

Detection Summary

This detection identifies PowerShell encoded command execution on a Windows endpoint. Encoded PowerShell commands can be used by attackers to hide command content and make malicious activity harder to review. Although encoded commands may also be used by legitimate administrators or automation tools, they are commonly associated with suspicious PowerShell activity.

Field	Value
Detection ID	DET-001
Detection Name	PowerShell Encoded Command Execution
Custom Rule ID	100100
MITRE ATT&CK Technique	T1059.001
Tactic	Execution
Log Source	Sysmon / Windows Process Telemetry
Status	Detected

Test Method

A harmless command was encoded and executed through PowerShell. The goal was not to run malicious code, but to generate realistic telemetry showing the use of the `-EncodedCommand` parameter.

The test command executed a simple identity check such as `whoami`, encoded in Base64 and executed through PowerShell.

Detection Value

This detection is valuable because PowerShell is frequently used in both legitimate administration and attacker activity. Detecting encoded PowerShell execution helps analysts identify commands that may be intentionally obfuscated.

This detection demonstrates visibility into:

- PowerShell process execution
- Command-line arguments
- Encoded command usage
- Sysmon process telemetry
- MITRE ATT&CK execution behavior

Analyst Triage Considerations

When reviewing this alert, an analyst should check:

- Which user executed PowerShell
- What parent process launched PowerShell
- Whether the encoded command can be decoded
- Whether the command was expected
- Whether the endpoint is used for administration
- Whether similar alerts occurred around the same time

False-Positive Considerations

Possible false positives include:

- Administrative scripts
- Software deployment tools
- Endpoint management tools
- IT automation tasks

The detection should not be blindly suppressed. Instead, known approved automation should be documented and reviewed before any allowlisting decision.

Evidence

Evidence collected for this detection includes:

- Wazuh dashboard alert screenshot
- Sanitized alert JSON
- MITRE ATT&CK mapping screenshot where available

DET-002: Local User Account Creation

Detection Summary

This detection identifies local user account creation on a Windows endpoint. Local account creation can be legitimate, but it can also be used by attackers for persistence after initial access.

Field	Value
Detection ID	DET-002
Detection Name	Local User Account Creation
Custom Rule ID	100101
MITRE ATT&CK Technique	T1136.001
Tactic	Persistence
Log Source	Windows Security Logs
Status	Detected

Test Method

A temporary local user account was created inside the Windows 10 lab VM. The account was created only for validation purposes and removed after the test.

The test simulated a persistence-related behavior without causing harm to the environment.

Detection Value

This detection is valuable because unexpected local account creation can indicate that an attacker is attempting to maintain access to a compromised endpoint.

This detection demonstrates visibility into:

- Local account creation
- Windows Security events
- Account manipulation behavior
- Persistence-related activity
- MITRE ATT&CK local account creation technique

Analyst Triage Considerations

When reviewing this alert, an analyst should check:

- Who created the account
- What account was created
- Whether the account name looks suspicious
- Whether the account was added to privileged groups
- Whether the account logged in after creation
- Whether the action was approved

False-Positive Considerations

Possible false positives include:

- Helpdesk support activity
- Local administrator maintenance
- Lab testing
- Temporary account creation for troubleshooting

Tuning should be based on creator account, endpoint role, maintenance window, and whether account creation is expected on that system.

Evidence

Evidence collected for this detection includes:

- Wazuh dashboard alert screenshot
- Sanitized alert JSON
- MITRE ATT&CK mapping screenshot where available

DET-003: Windows Service Creation / Modification

Detection Summary

This detection identifies Windows service creation or modification activity. Windows services are commonly used by legitimate software, but attackers may also abuse services for persistence, execution, and privilege escalation.

Field

Value

Detection ID	DET-003
Detection Name	Windows Service Creation / Modification
Custom Rule ID	100102
MITRE ATT&CK Technique	T1543.003
Tactic	Persistence / Privilege Escalation
Log Source	Windows Service / Process Telemetry
Status	Detected

Test Method

A temporary Windows service was created using `sc.exe` inside the Windows 10 lab VM. The service was used only to generate telemetry and was deleted after validation.

This test simulated service-based persistence behavior in a safe way.

Detection Value

This detection is valuable because service creation is a common technique for persistence on Windows systems. Attackers may create services that execute malicious binaries or scripts automatically.

This detection demonstrates visibility into:

- Service creation behavior
- Windows persistence techniques
- Service binary path review
- Windows process and service telemetry
- MITRE ATT&CK Windows service technique

Analyst Triage Considerations

When reviewing this alert, an analyst should check:

- Service name
- Service binary path
- Account that created the service
- Parent process
- Whether the service was started
- Whether the binary path is suspicious
- Whether the action matches known software installation activity

False-Positive Considerations

Possible false positives include:

- Software installation
- Driver updates
- Security agent updates
- Endpoint management tools
- Windows maintenance activity

Tuning should focus on suspicious service names, unusual binary paths, services created from temporary directories, or services created outside approved change windows.

Evidence

Evidence collected for this detection includes:

- Wazuh dashboard alert screenshot
- Sanitized alert JSON
- MITRE ATT&CK mapping screenshot where available

DET-009: EICAR Malware-Test File Detection

Detection Summary

This detection identifies Windows Defender detection or remediation of the EICAR malware-test file. EICAR is a safe anti-malware test file designed to trigger antivirus detection without using real malware.

Field	Value
Detection ID	DET-009
Detection Name	EICAR Malware-Test File Detection
Custom Rule ID	100108
MITRE ATT&CK Technique	T1204.002
Tactic	Execution

Log Source	Windows Defender Operational Logs
Status	Detected

Test Method

The EICAR test file was created in a controlled lab directory on the Windows 10 endpoint. Windows Defender detected or remediated the file, and the related event was forwarded into Wazuh.

No real malware was executed. The purpose of this test was to validate that Defender malware-detection telemetry reaches Wazuh.

Detection Value

This detection is valuable because antivirus or endpoint protection alerts should be visible in the SIEM. Defender detections can provide important context during incident response and malware triage.

This detection demonstrates visibility into:

- Windows Defender alerts
- Malware-test telemetry
- Defender Operational logs
- SIEM forwarding of endpoint protection events
- Safe malware simulation using EICAR

Analyst Triage Considerations

When reviewing this alert, an analyst should check:

- File path
- Threat name
- Defender action taken
- User involved
- Whether the detection was expected testing
- Whether similar detections occurred on other endpoints

False-Positive Considerations

EICAR itself is test activity. However, real Defender malware detections should be treated seriously. In production, analysts should separate known lab testing from unexpected malware detections.

Possible false positives include:

- Antivirus validation
- SOC testing
- Purple-team exercises
- Lab demonstrations

Evidence

Evidence collected for this detection includes:

- Wazuh dashboard alert screenshot
- JSON view screenshot

DET-010: Atomic Red Team Framework Activity

Detection Summary

This detection identifies Atomic Red Team framework activity on a Windows endpoint. Atomic Red Team is commonly used by blue teams, detection engineers, and purple teams to validate MITRE ATT&CK-aligned detection coverage.

Although Atomic Red Team is a legitimate testing framework, unexpected execution outside an approved lab or testing window may require investigation.

Field	Value
Detection ID	DET-010
Detection Name	Atomic Red Team Framework Activity
Custom Rule ID	100109
MITRE ATT&CK Technique	T1059.001
Tactic	Execution
Log Source	Sysmon Event ID 1
Status	Detected

Test Method

Atomic Red Team was installed and used in a controlled manner to generate framework-related telemetry. The test used a safe display/validation command rather than destructive payload execution.

The purpose was to confirm that Atomic Red Team framework activity could be detected through Sysmon process telemetry and reviewed inside Wazuh.

Detection Value

This detection is valuable because it shows the ability to detect adversary-emulation framework usage. In professional environments, Atomic Red Team activity should normally occur only on approved testing systems and during approved testing windows.

This detection demonstrates visibility into:

- PowerShell-based framework activity
- Atomic Red Team usage
- Adversary-emulation validation
- Sysmon Event ID 1 process telemetry
- MITRE ATT&CK-aligned testing behavior

Analyst Triage Considerations

When reviewing this alert, an analyst should check:

- Which user executed the command
- Which host generated the alert
- Whether Atomic Red Team testing was approved
- Which technique was referenced
- Whether additional tests were executed
- Whether the endpoint is a lab or production system

False-Positive Considerations

Possible false positives include:

- Approved detection engineering tests
- Purple-team exercises
- SOC validation activities
- Training labs

Tuning should not suppress all Atomic Red Team usage globally. Instead, approved tester accounts, lab endpoints, and testing windows should be documented.

7. Evidence Handling and Redaction

Evidence handling was an important part of this project. Screenshots and JSON files were collected to prove that detections fired successfully, but sensitive lab details were redacted before public use.

Sensitive details that should be removed or anonymized include:

- Real usernames
- Real hostnames
- Private IP addresses where not necessary
- Agent IDs
- Personal file paths
- API keys, tokens, or passwords

Example redactions:

Original Type	Replacement
Real Windows username	lab_user
Real Windows hostname	WIN10-LAB
Wazuh agent name	windows-endpoint
Wazuh server name	wazuh-server
Private IP address	192.168.56.101

Evidence should be separated into curated evidence and archived supporting screenshots. The curated evidence folder should contain only the strongest proof screenshots. Raw setup, testing, and troubleshooting screenshots should be archived separately.

This improves the professionalism of the project and makes it easier for reviewers to understand the detection results.

8. False-Positive Tuning Strategy

The project includes false-positive analysis because detection engineering is not only about creating alerts. A good detection must also explain when the alert may be legitimate and how an analyst should tune or interpret it.

The tuning strategy follows these principles:

1. Do not suppress alerts only because they can be legitimate.
2. Review user, host, parent process, command line, and endpoint role.
3. Separate lab activity from production assumptions.
4. Document known false-positive sources.
5. Tune only after confirming expected behavior.
6. Preserve high-value detections such as service creation, Defender detections, and suspicious PowerShell usage.

Examples:

- Encoded PowerShell may be used by legitimate automation, but unexpected encoded commands should be reviewed.
 - Local user creation may be legitimate during maintenance, but unexpected local accounts can indicate persistence.
 - Windows service creation may occur during software installation, but suspicious service paths or unusual parent processes increase risk.
 - EICAR should be treated as test activity, but real Defender malware detections should remain high priority.
 - Atomic Red Team activity is normal in a lab, but unexpected use in production should be investigated.
-

9. Analyst Triage Approach

Each detection includes analyst-focused triage logic. The goal is to show how an alert would be reviewed in a SOC environment.

General triage questions include:

- What happened?
- Which host generated the alert?
- Which user was involved?
- What command or process triggered the detection?

- Is the behavior expected?
- Does the activity map to a known MITRE ATT&CK technique?
- Are there related alerts from the same host?
- Does the alert require escalation?

This makes the project stronger because it shows more than rule writing. It shows how the alert would be interpreted and investigated.

10. Project Limitations

This project was built in a controlled local lab and has several limitations.

The main limitations are:

- The detections were validated on a small lab environment, not a production enterprise network.
- The project focuses on five validated detections rather than broad coverage.
- Some detections depend on telemetry being enabled, such as Sysmon or Windows Defender Operational logs.
- False-positive tuning was documented conceptually but not tested across a large enterprise environment.
- The lab does not include Active Directory, centralized identity telemetry, EDR enrichment, or large-scale correlation.
- The project does not claim to detect every possible variation of the mapped techniques.

These limitations are acceptable because the goal of the project is to demonstrate the detection engineering process, not to claim production-ready enterprise coverage.

11. Lessons Learned

This project provided several important lessons about Windows detection engineering and SOC workflows.

First, detection engineering requires proof. Writing a rule is not enough. A detection should be tested, validated, documented, and supported with evidence.

Second, telemetry quality matters. Sysmon and Windows Defender logs significantly improve visibility into endpoint behavior. Without the correct telemetry source, a rule may not fire even if the logic is correct.

Third, MITRE ATT&CK mapping makes detections easier to explain. Mapping each detection to a technique helps connect low-level logs to attacker behavior.

Fourth, false-positive documentation is part of detection quality. A detection that fires frequently without context may create noise. A useful detection should explain what may be suspicious, what may be legitimate, and what an analyst should review.

Fifth, safe testing scope matters. EICAR and Atomic Red Team make it possible to validate security monitoring without using real malware or harmful activity.

Sixth, evidence organization improves credibility. Clean screenshots, sanitized JSON files, validation matrices, and triage notes make the project easier for recruiters, analysts, and interviewers to understand.

Finally, honest scope is more important than inflated claims. A clean project with five validated detections is stronger than a larger project that claims detections without proof.

12. Future Improvements

This project can be expanded in future versions by adding more detections only after full validation and documentation.

Potential future detections include:

- Multiple failed Windows logons
- User added to local Administrators group
- Scheduled task creation
- Windows Security log cleared
- Suspicious certutil execution
- PsExec execution
- ProcDump usage against a safe non-sensitive process
- mshta suspicious script execution

Future technical improvements could include:

- Adding Wazuh logtest validation screenshots
- Adding Sigma rule equivalents for each detection
- Adding MITRE ATT&CK Navigator coverage mapping
- Adding GitHub Actions to check Markdown and XML quality

- Creating one folder per detection with rule, command, screenshot, and JSON evidence
- Adding severity rationale for each rule
- Adding response recommendations for each alert
- Adding additional sanitized JSON examples for DET-009 and DET-010

Each future detection should include:

1. Custom rule
2. Test command
3. MITRE mapping
4. Screenshot evidence
5. JSON evidence where possible
6. False-positive notes
7. Analyst triage guidance

13. Final Summary

This project demonstrates a realistic Windows detection engineering workflow using Wazuh, Sysmon, Windows Security logs, Windows Defender telemetry, Atomic Red Team, and MITRE ATT&CK mapping.

The strongest part of the project is that it does not only show tool installation. It shows the process of building, testing, validating, documenting, and explaining detections in a way that aligns with SOC analyst and junior detection engineering responsibilities.

The final validated detection scope is:

Detection ID	Detection Name
DET-001	PowerShell Encoded Command Execution
DET-002	Local User Account Creation
DET-003	Windows Service Creation / Modification
DET-009	EICAR Malware-Test File Detection
DET-010	Atomic Red Team Framework Activity